

SysHospitalar

Comunicação assíncrona e
processamento em lote

AUTOR	DISCIPLINA
Pedro Carrara	Arquiteturas Avançadas de Software com Microserviços e Spring Framework

Relatório técnico de implementação, testes e demonstração

Neste projeto, desenvolvi uma API para o gerenciamento de pacientes, médicos, enfermeiros e atendimentos hospitalares. A evolução foi organizada em etapas: separação de responsabilidades, extração do serviço de atendimentos, configuração externa e execução em containers.

Na Etapa 4, acrescentei a confirmação de atendimento por mensageria e a importação de pacientes por CSV com Spring Batch. A escolha dos mecanismos considera o momento em que o resultado é necessário e se a operação trata um registro ou um conjunto de informações.

Comunicação	Processamento	Validação
REST e RabbitMQ	CSV e chunks de 10	17 testes aprovados

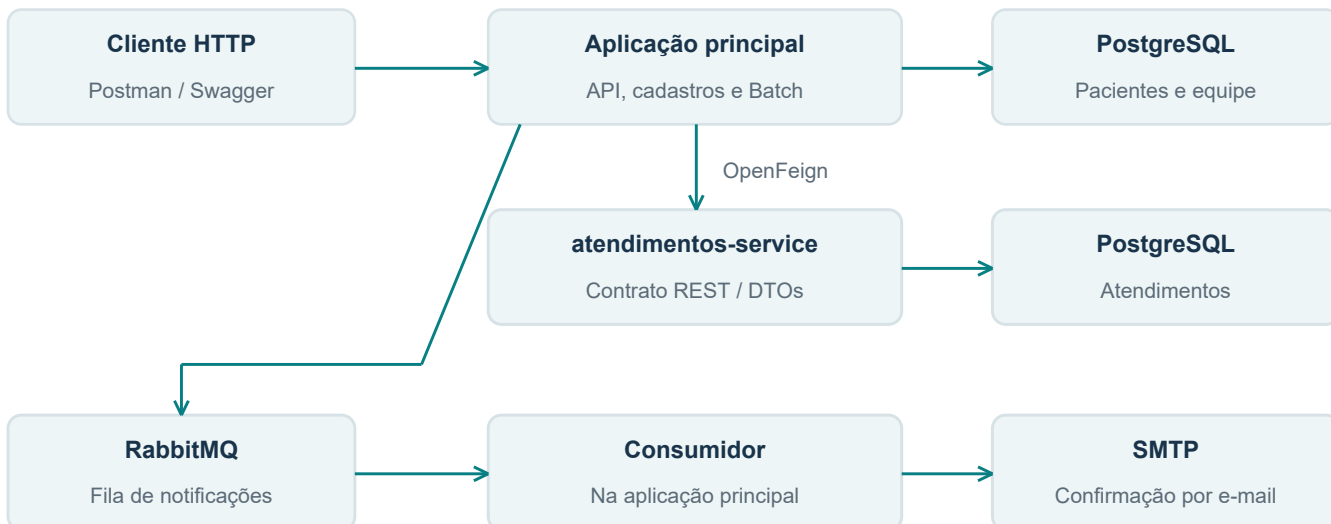
Organização do relatório

Arquitetura e responsabilidades (p. 2); mensageria (p. 3); Spring Batch (p. 4); evidências automatizadas (p. 5); execução (p. 6); demonstração (p. 7); reflexão arquitetural (p. 8); limites e fontes (p. 9).

As evidências automatizadas correspondem à execução de 5 de outubro de 2026. Os cenários manuais são descritos como procedimentos e resultados esperados; não são apresentados como capturas de uma execução realizada.

Arquitetura e responsabilidades

Mantive uma aplicação principal e extraí apenas a responsabilidade de atendimentos para um serviço independente. O Config Server é um componente de infraestrutura. Bancos e RabbitMQ complementam o ambiente de execução.



Config Server fornece as configurações das APIs. CSV é processado na aplicação principal.

Componente	Responsabilidade
Aplicação principal	Cadastros de pacientes e equipe; validações locais; integração HTTP; produtor e consumidor de notificações; importação Batch.
atendimentos-service	Persistência e operações de atendimentos. Referencia paciente e médico por identificador, usando banco próprio.
Config Server	Centralização de configurações por nome da aplicação e profile. Segredos são fornecidos por variáveis de ambiente.

Fluxo síncrono preservado

O cliente acessa o controller, que delega ao service. Nos cadastros locais, o service utiliza repositories JPA. Para atendimentos, o service valida paciente e médico, chama o serviço por OpenFeign e enriquece a resposta com os nomes locais. Os contratos HTTP utilizam DTOs.

A aplicação principal não acessa o banco do serviço de atendimentos. Uma falha de comunicação é convertida em resposta controlada, como HTTP 503, sem apresentar o endereço interno ou a exceção de conexão ao cliente.

ETAPA 4 / 03

Notificação assíncrona

Operação escolhida: enviar uma confirmação ao paciente após o registro do atendimento.

Produção, fila e consumo

Após a confirmação da persistência pelo serviço remoto, a aplicação principal publica um `AtendimentoCriadoEvento`. O produtor usa `RabbitTemplate`. O consumidor recebe o evento por um listener `RabbitMQ` e delega a montagem e o envio ao serviço de e-mail. A resposta HTTP 201 não aguarda a conclusão do SMTP.

```
Produtor -> atendimentos.exchange -> fila -> consumidor -> SMTP
Routing key: atendimento.criado
Fila: notificacao-email.atendimento-criado
DLQ: notificacao-email.atendimento-criado.dlq
```

Contrato da mensagem

O evento contém apenas identificação, destinatário e data do atendimento. O conteúdo abaixo é um exemplo ilustrativo, não uma captura de mensagem real. Não são enviados diagnóstico, prontuário ou detalhes clínicos.

```
{
  "eventoId": "f6ccde56-cf95-4d0c-b183-bc4968b89f50",
  "tipoEvento": "ATENDIMENTO_CRIADO",
  "atendimentoId": 42,
  "pacienteNome": "Paciente Demonstracao",
  "pacienteEmail": "paciente@example.com",
  "dataHoraAtendimento": "2026-10-10T14:30:00"
}
```

Situação	Comportamento
Consumidor desativado	Com o broker disponível, a mensagem aguarda na fila; após a reativação, o consumidor a processa.
Falha SMTP	Há três tentativas: a inicial e duas novas, após esperas de 2 e 4 segundos. Se persistir a falha, a mensagem segue para a DLQ.
Falha de publicação	O produtor trata a falha do broker para preservar o atendimento criado. A notificação pode ser perdida, pois não há transacional outbox.

Produtor e consumidor estão na mesma aplicação, mas são componentes separados. A variável `NOTIFICACOES_CONSUMIDOR_ATIVO` permite desativar o listener e manter API e produtor disponíveis.

ETAPA 4 / 04

Importação com Spring Batch

Funcionalidade escolhida: importar múltiplos pacientes de um arquivo CSV para o banco principal.

```
CSV -> ItemReader -> ItemProcessor -> ItemWriter -> PostgreSQL  
Job: importacaoPacientesJob  
Step: importarPacientesStep  
Chunk: 10
```

Etapa	Implementação
Leitura	FlatFileItemReader e PacienteCsvLineMapper leem UTF-8, pulam o cabeçalho e interpretam sete colunas.
Processamento	PacienteImportacaoProcessor normaliza campos, valida regras e filtra duplicidades de CPF ou e-mail no banco e no arquivo.
Escrita	JpaItemWriter grava os registros válidos no PostgreSQL pertencente à aplicação principal.

Regras aplicadas aos registros

Campo	Regra
Nome, CPF e telefone	Compactar espaços do nome; retirar pontuação do CPF e caracteres não numéricos do telefone. CPF deve ter 11 dígitos.
Data, sexo e e-mail	Data de nascimento não futura; sexo M ou F; e-mail normalizado para minúsculas e validado.
Ativo e duplicidades	Ativo vazio assume true. CPF ou e-mail já conhecidos levam à filtragem do registro.

Entrada, execução e resultado

O POST /batch/pacientes/importacoes recebe multipart/form-data no campo arquivo e retorna 202 com executionId e Location. O GET no mesmo caminho seguido de /{executionId} apresenta estado, horários e contadores de lidos, gravados, filtrados e ignorados.

O arquivo deve ter extensão .csv e o cabeçalho abaixo. O limite configurado é de 2 MB para o arquivo e também para a requisição completa, incluindo o envelope multipart.

```
nome,cpf,dataNascimento,sexo,telefone,email,ativo
```

Cada chunk considera até dez itens lidos; itens filtrados não chegam ao writer. O último bloco pode ser menor. Até 100 erros estruturais podem ser ignorados; o seguinte faz o Job falhar. Blocos já confirmados podem permanecer gravados.

ETAPA 4 / 05

Evidências automatizadas

Resultados extraídos dos relatórios JUnit XML gerados pelo Maven Surefire em 5 de outubro de 2026.

Grupo de testes	Casos	Falhas / erros / ignorados
Ordem e condições da publicação	3	0 / 0 / 0
Fila, consumo, retry e DLQ	3	0 / 0 / 0
Conteúdo do e-mail e falha do broker	2	0 / 0 / 0
Persistência, regras e Batch	7	0 / 0 / 0
Persistência de atendimento	1	0 / 0 / 0
Inicialização do Config Server	1	0 / 0 / 0
TOTAL	17	0 / 0 / 0

Comportamentos efetivamente exercitados

Mensageria: o teste interrompe o listener, publica uma mensagem, observa a mensagem na fila e confirma ausência de envio. Depois, reinicia o listener e verifica a chamada de envio. Outro cenário provoca falha SMTP, verifica três chamadas e encontra uma mensagem real na DLQ.

Batch: um CSV de teste com três registros termina como COMPLETED, com 3 lidos, 1 gravado e 2 filtrados. São conferidos nome, telefone e e-mail normalizados. Outro cenário com 101 linhas malformadas termina como FAILED e preserva o arquivo temporário.

API: os testes verificam aceite HTTP 202, cabeçalho Location, consulta do resultado e HTTP 404 para execução inexistente. Também há verificação de duplicidade de CPF e e-mail.

Condições e alcance da evidência

As três execuções Maven terminaram com BUILD SUCCESS. PostgreSQL e RabbitMQ foram iniciados pelo Testcontainers; JavaMailSender e a chamada remota no teste unitário foram simulados. Portanto, esses testes não comprovam recebimento pelo Gmail nem um único fluxo HTTP completo com todas as aplicações.

Ambiente observado: Java 23.0.2; alvo de compilação Java 21. A suíte de atendimentos apresentou aviso de encerramento da JVM pelo Surefire após 30 segundos, apesar do teste e do build aprovados. Os relatórios consultados estão em target/surefire-reports de cada aplicação.

ETAPA 4 / 06

Como executar o projeto

Roteiro para executar todos os componentes localmente pelo profile prod do Docker Compose.

Pré-requisitos e configuração

Utilize Docker Desktop ou Docker Engine com Compose e acesso à internet para baixar as imagens. Para executar os testes fora dos containers, utilize JDK 21 e o Maven Wrapper. Os comandos abaixo partem da raiz do repositório, em PowerShell.

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
```

Edite .env para configurar as senhas dos bancos e do RabbitMQ. Para o envio real escolhido neste projeto, configure MAIL_USERNAME, MAIL_PASSWORD e MAIL_FROM com uma conta de demonstração. Nunca inclua esse arquivo ou senhas no PDF ou no repositório.

Inicialização e verificação

```
docker compose --profile prod config --quiet
docker compose --profile prod up --build -d --wait
docker compose --profile prod ps
```

Resultado esperado: dois PostgreSQL, RabbitMQ, Config Server, atendimentos-service e aplicação principal saudáveis. São seis componentes, incluindo infraestrutura. A validação config --quiet verifica a configuração sem mostrar credenciais; não executa o sistema.

Interface	Endereço local
API principal / Swagger	http://localhost:8080/swagger-ui.html
Atendimentos / Swagger	http://localhost:8081/swagger-ui.html
Painel RabbitMQ	http://localhost:15672
Config Server	http://localhost:8888

Testes e encerramento

```
.\mvnw.cmd test
.\mvnw.cmd -f atendimentos-service\pom.xml test
.\mvnw.cmd -f config-server\pom.xml test
docker compose --profile prod down
```

Os testes de integração exigem Docker disponível; testes ignorados não comprovam os cenários de integração. O comando down preserva os volumes. Não acrescente -v para uma demonstração que deva conservar os dados.

O profile dev da aplicação principal usa create-drop. Dev e prod compartilham os volumes do Compose: não utilize dev sobre dados que deseja manter. Para esta demonstração, mantenha o profile prod.

ETAPA 4 / 07

Roteiro de demonstração

Procedimentos reproduzíveis e resultados esperados. Esta seção não representa capturas de uma demonstração manual concluída.

1. Cadastro e comunicação REST

Pelo Swagger principal, crie um paciente fictício com e-mail sob seu controle e um médico. Use os IDs retornados no POST /atendimentos. Espere HTTP 201 e consulte o mesmo atendimento pela API principal e pelo serviço na porta 8081. Em prod, não presuma IDs iguais a 1.

2. Mensagem aguardando na fila

```
$env:NOTIFICACOES_CONSUMIDOR_ATIVO="false"
docker compose --profile prod up -d --no-deps `
  --force-recreate aplicacao-principal
```

Aguarde a aplicação ficar saudável e crie outro atendimento. No painel RabbitMQ, observe a fila notificacao-email.atendimento-criado: Consumers deve ser 0 e Ready deve aumentar. O broker e o produtor permanecem ativos. Registre o ID do atendimento e o contador, sem expor dados pessoais.

3. Retomada e falha controlada

```
$env:NOTIFICACOES_CONSUMIDOR_ATIVO="true"
docker compose --profile prod up -d --no-deps `
  --force-recreate aplicacao-principal
docker compose --profile prod logs -f aplicacao-principal
```

Espere o processamento, a redução de Ready e o log de sucesso. No envio real, confira a caixa de e-mail. Uma falha SMTP persistente produz três tentativas e DLQ; o guia de testes descreve como provocar a falha e restaurar a configuração. Remova a sobrescrita da variável do terminal ao terminar.

4. Upload e consulta do Batch

```
curl.exe -i -F "arquivo=@src/main/resources/batch/pacientes-exemplo.csv" `
  http://localhost:8080/batch/pacientes/importacoes
```

Copie o executionId da resposta 202 e consulte GET /batch/pacientes/importacoes/{executionId} até o estado final. Consulte GET /pacientes para verificar os registros. Sem CPFs ou e-mails já coincidentes, o CSV de exemplo deve produzir 5 lidos, 2 gravados, 3 filtrados e 0 ignorados. Esses são resultados esperados do exemplo; o teste automatizado usa outro CSV, descrito na página 5.

Evidências a registrar na apresentação: serviços saudáveis; HTTP 201; fila em espera; consumo após retomada; HTTP 202 e estado final do Batch. A coleção Postman da Etapa 4 e os exemplos completos estão no repositório.

Reflexão arquitetural da Etapa 4

1. Qual operação foi escolhida para comunicação assíncrona?

Escolhi a confirmação de atendimento por e-mail. Após a persistência, a aplicação publica um evento no RabbitMQ e o consumidor realiza o envio ao paciente.

2. Por que a operação pode ocorrer após a requisição original?

O atendimento já existe quando a mensagem é publicada. O e-mail informa o resultado e não determina o sucesso do cadastro; por isso, o cliente não precisa aguardar o SMTP.

3. O que acontece se o consumidor estiver indisponível?

Com o broker disponível, a mensagem permanece aguardando na fila. Quando o consumidor é reativado, o processamento pode continuar. Essa espera diferencia o fluxo da resposta imediata exigida em uma chamada REST.

4. Qual funcionalidade foi escolhida para processamento em lote?

Escolhi a importação de pacientes por CSV. O Job lê múltiplos registros, valida e normaliza os campos e grava os pacientes válidos no banco da aplicação principal.

5. Por que essa funcionalidade é adequada para Batch?

Vários registros precisam passar pelo mesmo fluxo de leitura, processamento e escrita. O Spring Batch organiza o trabalho em chunks de 10 e fornece estado da execução, transações por bloco e contadores.

6. Quando utilizar REST, mensageria ou Batch no projeto?

Mecanismo	Situação no SysHospitalar	Motivo
REST	Cadastro e consulta; criação de atendimento no serviço independente.	É necessário obter uma resposta imediata.
Mensageria	Confirmação de atendimento por e-mail.	Componentes se comunicam de forma assíncrona; o envio pode ocorrer depois.
Batch	Importação de pacientes a partir de CSV.	Um conjunto de dados passa por processamento estruturado.

Limites, segurança e fontes

Segurança e execução em containers

O projeto utiliza Bean Validation, DTOs e tratamento centralizado de exceções previstas. As credenciais são externalizadas e .env é ignorado pelo Git. O Compose publica portas na interface local, e os Dockerfiles executam as aplicações com usuário sem privilégios de administrador.

Não há autenticação ou autorização implementadas nas APIs consultadas. Portanto, essas medidas não equivalem a controle de acesso completo ou segurança robusta para uso real. Uma implantação real exigiria definição de identidade, permissões, proteção de tráfego e operação dos segredos.

Dockerfiles, configurações externas, healthchecks, redes e volumes fornecem uma base de execução em containers. O ambiente prod do Compose é uma simulação acadêmica local: este relatório não comprova implantação em provedor de nuvem nem escalabilidade avaliada por testes de carga.

Limitações funcionais conhecidas

A notificação não possui transactional outbox, confirmação de publicação pelo broker ou idempotência persistente: podem ocorrer perda na publicação ou duplicação em reentregas. Não há reprocessamento automático da DLQ.

A importação admite uma execução ativa por vez. O teste atual não atravessa vários chunks. Arquivos de Jobs concluídos são removidos; os de falha são preservados no filesystem, mas não há volume de CSV no Compose. Uma queda abrupta pode deixar metadados de execução ativa sem endpoint de recuperação.

Fontes do relatório e material de apoio

O conteúdo foi conferido contra o enunciado fornecido, o README, as configurações e as classes de implementação e teste do projeto. A tabela da página 5 foi extraída dos relatórios Surefire das três aplicações, selecionando as classes atuais e excluindo um relatório antigo de setembro.

Material	Localização
Repositório	github.com/pedrocarrara12/sysHospitalar
Requisitos e demonstração	README.md; docs/etapas/etapa-4-assincrono-batch.md; docs/etapas/etapa-4-guia-testes.md
Dados e requisições	src/main/resources/batch/pacientes-exemplo.csv; postman/SysHospitalar-Etapa4.postman_collection.json

Uso acadêmico de IA

Boa noite, professor! Conforme permitido no enunciado, utilizei a IA como apoio durante o desenvolvimento do projeto. Pedi que ela atuasse como uma assistente de código, explicando as decisões, como implementar cada parte e o porquê das escolhas. Esse apoio me ajudou a aprender bastante ao longo do trabalho, buscando compreender o que estava construindo.

Também mantive no projeto alguns arquivos Markdown, como o AGENTS.md, com orientações e registros de como utilizei a ferramenta. Eles estão disponíveis caso o senhor queira conhecer melhor esse processo.